THE NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, LAHORE
CS-4031 Compiler Construction — Fall 2026

Lecture 4
Regular Expressions, Regular Definitions, and Extensions; Regular vs. Non-Regular Languages; Real-World Lexer Design and Flex

CLO(s) covered:	CLO 1
Dragon Book [DB]:	§3.3 (Specification of Tokens: regular expressions, regular definitions, extensions)
Other references:	[PP] Programming Language Pragmatics §3.3

Lecture Roadmap

1. **Regular expressions**: the formal, inductive notation for naming the regular languages defined last lecture, plus many worked examples.
2. **Regular definitions**: naming sub-expressions, culminating in DB's classic identifier and numeric-literal definitions.
3. **Extensions** (`+`, `?`, character classes) – convenience, not new power – and **regular vs. non-regular languages**: what a lexer's patterns fundamentally cannot express.
4. How real-world lexers extend this model, and how the regular expressions in this lecture map directly onto the syntax used by **Flex**, the lexer-generator tool this course uses.

Lecture **??** defined the regular *languages* themselves, as sets of strings. Today gives us the *notation* – regular expressions – for writing them down, and by the end of this lecture you will be able to read and write the formal notation a real lexer specification is built from.

4.1 Regular Expressions

[DB] A **regular expression** is a compact notation for describing a language. DB defines regular expressions *inductively*: a small set of base cases, plus rules for combining smaller regular expressions into bigger ones – deliberately mirroring, symbol for symbol, the definition of the regular languages themselves from §**??**.

Definition: Regular Expressions (Inductive Definition)

Over alphabet Σ:

- **Base case 1:** ε is a regular expression; $L(\varepsilon) = \{\varepsilon\}$.
- **Base case 2:** for each symbol $a \in \Sigma$, a is a regular expression; $L(a) = \{a\}$.
- **Inductive case (union):** if r and s are regular expressions, so is $r \mid s$, with $L(r \mid s) = L(r) \cup L(s)$.
- **Inductive case (concatenation):** if r and s are regular expressions, so is rs, with $L(rs) = L(r)L(s)$.
- **Inductive case (closure):** if r is a regular expression, so is r^*, with $L(r^*) = (L(r))^*$.
- **Parenthesization:** if r is a regular expression, so is (r), with $L((r)) = L(r)$ – used purely to control grouping/precedence.

Here $L(r)$ denotes "the regular language named by regular expression r" – notice every base case

and inductive case above produces a regular language in exactly the sense of §**??**, and nothing else, matching that definition rule for rule.

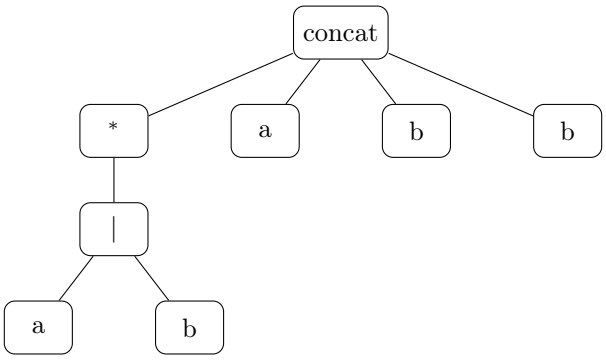
Operator Precedence

When parentheses are omitted, DB fixes a precedence, highest to lowest: **closure** (*) binds tightest, then **concatenation**, then **union** (|) binds loosest. So $ab \mid c$ means $(ab) \mid c$, *not* $a(b \mid c)$; and $a \mid bc^*$ means $a \mid (b(c^*))$.

4.1.1 Worked Example: Building Up a Regular Expression

DB's running example: the language of all strings of `a`'s and `b`'s that *end in* `abb`. Let's build the regular expression piece by piece.

Sub-expression	What it describes
$a \mid b$	a single `a` *or* a single `b`
$(a \mid b)^*$	*any* string of `a`'s and `b`'s (including ε)
$(a \mid b)^*abb$	any string of `a`'s and `b`'s, **followed by** the literal suffix `abb` – i.e. exactly the strings that *end in* `abb`



Syntax tree for $(a \mid b)^*abb$: the root concatenates four pieces — the starred union, then three literal symbols.

Example: Trace: Does `"babb"` Match?

`babb` = `b` (matched by the $(a|b)^*$ part, one repetition) followed by `a`, `b`, `b` (the literal suffix). Yes, it matches. Does `"abab"` match? No – it does not end in the literal substring `abb` (it ends in `bab`).

Exam Focus

Given an informal language description ("strings that start with …", "strings with an even number of …", "strings that don't contain …"), be able to construct the regular expression, and vice versa: given a regular expression, describe its language in English and list 2–3 example strings it does and does not match. This two-way translation is the most common exam pattern for this topic.

4.1.2 More Worked Examples

Language (in English)	Regular Expression	Matches / Doesn't Match
Binary strings with an even number of 0's	$1^*(01^*01^*)^*$	Matches `110011`; doesn't match `100`
Strings over $\{a, b\}$ containing at least one `a`	$(a \mid b)^* a (a \mid b)^*$	Matches `bba`; doesn't match `bbb`
Strings over $\{a, b\}$ *not* containing the substring `aa`	$b^*(ab^+)^*a^?$	Matches `ababb`; doesn't match `baab`
C-style single-line comment (using Flex's `.` for "any character except newline"[1])	`"//" .*`	Matches `// TODO fix this`; doesn't match a two-line comment

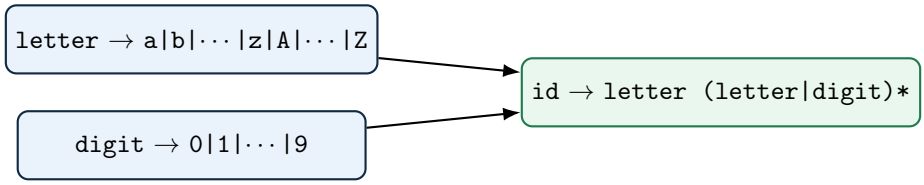
4.2 Regular Definitions

[DB] Writing out a complex regular expression as one giant unreadable formula is painful. A **regular definition** lets us name intermediate regular expressions and reuse those names in later definitions — exactly like defining helper variables in ordinary algebra.

Definition: Regular Definition

A regular definition is a sequence $d_1 \to r_1,\ d_2 \to r_2,\ \ldots,\ d_n \to r_n$ where each d_i is a new name, and each r_i is a regular expression over $\Sigma \cup \{d_1, \ldots, d_{i-1}\}$ – that is, r_i may use the alphabet *and* any name defined *before* it, but never itself or a later name (no recursive or forward references).

4.2.1 Worked Example: Identifiers

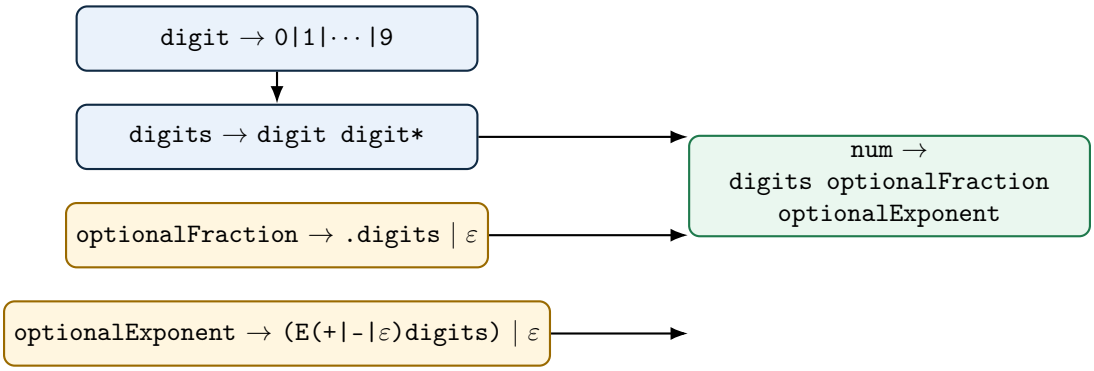


This is exactly $L(L \cup D)^*$ from the worked example in §**??**, now written as a named, reusable regular definition – and it is the pattern for token `ID` in almost every C-family language.

4.2.2 Worked Example: Unsigned Numbers (DB's Classic Definition)

Numbers like `5280`, `0.01`, `6.336E4`, or `1.89E-4` all need to be matched by a single token pattern. DB builds this up from four named pieces:

[1] Writing this with only the six core operators from §4.1 would require an enormous union of every symbol in Σ except newline. Real tools sidestep this with a dedicated wildcard; Flex's is `.`, and its full regex syntax – including this quoted-literal `"//"` convention – is covered in §4.6.5.



Example: Tracing `num` Against `6.336E4`

`digits` matches `6`; `optionalFraction` matches `.336` (the `.digits` branch); `optionalExponent` matches `E4` (the ε-sign branch of `(E(+|-|ε)digits)`, since no `+`/`-` appears). Concatenated: `6` + `.336` + `E4` = `6.336E4`. For a plain integer like `5280`, both `optionalFraction` and `optionalExponent` take their ε branch and contribute nothing.

Exam Focus

Be able to reproduce this exact four-piece `num` definition from memory, and trace it against a given number (integer, decimal, or scientific-notation) the way the example above does. This is DB's single most frequently examined regular-definition example.

4.3 Extensions of Regular Expressions

[beyond DB] Modern regex notation (and DB §3.3.5) adds a few extra operators. Formally, none of these add any expressive power beyond plain union/concatenation/closure — they are **syntactic sugar**, purely for convenience and readability.

Extension	Meaning	Equivalent to Core Regex
r^+	one or more repetitions	rr^*
$r^?$	zero or one occurrence (optional)	$r \mid \varepsilon$
$[abc]$	character class: any one of `a`, `b`, `c`	$a \mid b \mid c$
$[a\text{-}z]$	character range: any letter from `a` to `z`	$a \mid b \mid \cdots \mid z$
$[\hat{}a]$	negated class: any symbol *except* `a`	union of all $x \in \Sigma \setminus \{a\}$

Exam Focus

Know that $r^+ \equiv rr^*$ and $r^? \equiv (r \mid \varepsilon)$ *exactly* — a common trick question asks you to rewrite an extended-notation regex using only the three core operators (union, concatenation, closure), which is precisely this substitution.

4.4 Regular vs. Non-Regular Languages

[DB] Not every language can be described by a regular expression. This matters enormously for compiler design: it is exactly *why* a single lexer pass is not enough, and why we need an entirely different, more powerful formalism (context-free grammars, covered once the course reaches parsing) for parsing.

The Central Limitation: No Counting

A regular expression (equivalently, a finite automaton, covered once the course reaches finite automata) has no way to *count* an unbounded quantity and later check that count against something else. It can verify local, bounded patterns, but it cannot verify *global balance*. This single limitation explains every classic example of a non-regular language.

Language	Regular?	Why
Identifiers: letter followed by letters/digits	**Yes**	Purely local: check each character against a fixed small set of rules, no counting needed.
Fixed-format numeric literals	**Yes**	Same reason – bounded structure, no unbounded counting or matching required.
Balanced parentheses, e.g. $\{a^n b^n \mid n \geq 0\}$	**No**	Requires counting how many `(` were seen and demanding *exactly* that many `)` later — an unbounded count with no fixed upper limit.
Matching HTML/XML open and close tags	**No**	Same reason, at the level of tags instead of characters – and worse, tags can *nest*, requiring a stack, not just a counter.
Palindromes over $\{a, b\}$	**No**	Requires comparing the first half of the string against the reverse of the second half – again, unbounded memory of what came before.

Example: Why Finite Automata Can't Count: A Concrete Illustration

Suppose you tried to build a finite automaton for balanced parentheses up to nesting depth n. You would need at least one distinct state per depth level, $0, 1, 2, \ldots, n$, just to remember "how many unmatched `(` have I seen so far." Since parentheses in real programs can nest arbitrarily deep, no *fixed, finite* number of states suffices for *every* possible program – yet a finite automaton is required to have a fixed number of states, decided in advance. This is the intuitive heart of the **pumping lemma for regular languages** from your Automata Theory prerequisite: pump the middle of a long enough matched string and you break the balance, proving no finite automaton can recognize it.

Bridging Idea – Why This Matters for the Rest of the Course

- Regular expressions/finite automata are the right tool for tokens: identifiers, numbers, keywords, operators – all locally-checkable, bounded patterns.
- Nested and recursive structure – matched parentheses, balanced braces, nested expressions, `if`/`else` matching – is fundamentally *beyond* what regular expressions can express, no matter how cleverly you write them.
- This is precisely why the course introduces **context-free grammars** once parsing begins: a strictly more powerful formalism (using a stack, conceptually) that *can* count and match nested structure, and precisely why the compiler needs *two* separate formalisms — one per phase — instead of one universal one.

4.5 Real-World Lexer Design

[beyond DB] Everything so far in this lecture follows DB's simplified model fairly closely. Production lexers extend that model in several practical directions worth knowing about, even though we will not build any of them by hand in this course.

4.5.1 Handling Keywords: The Reserved-Word Trick

A naive lexer might need a separate hand-written check for every keyword (`if`, `while`, `return`, ...) before falling back to the generic identifier pattern. Production lexers instead use a much simpler trick: pre-load the symbol table with every reserved word *before* scanning begins, each marked with its own keyword token. The lexer then has only *one* pattern for anything "letter followed by letters/digits." When it matches such a lexeme, it looks the lexeme up in the symbol table: if an entry already exists (because it's a reserved word), the lexer returns *that* token (e.g. `WHILE`); if no entry exists, it's a genuine user identifier, so the lexer creates a new symbol-table entry and returns `ID`. One pattern, one lookup, no special-casing in the automaton itself.

4.5.2 The Maximal Munch Rule

When several patterns could all match a prefix of the remaining input, real lexers apply the **longest-match** ("maximal munch") rule: always take the *longest* lexeme that matches *any* pattern. This is why `<=` is scanned as a single `RELOP` token and not as `<` followed by `=`, and why `i` in `int` is not prematurely cut off as an identifier before the lexer has looked far enough ahead to see the rest of the word. C's `--` (decrement) versus `- -` (two unary minuses) and `->` (member access through a pointer) are further classic examples where getting the lookahead right matters. We formalize *why* this rule is needed, and how a scanner implements it efficiently, once we cover finite automata in the lectures ahead.

4.5.3 Numeric and String Literals in Practice

DB's `NUM` pattern is deliberately simplified. Real languages support numeric literal formats far beyond plain decimal integers, and each variant is a distinct lexical pattern the scanner must recognize:

Format	Example
Hexadecimal / octal / binary	`0x1F`, `0o17`, `0b1010`
Scientific notation	`6.022e23`, `1.5E-10`
Digit-group separators	`1_000_000` (Python, Java, Rust, C++14)
Type suffixes	`3.14f` (float), `100L` (long), `42u` (unsigned)
Complex/imaginary literals	`3+4j` (Python)

Each of these is its own regular pattern (or family of patterns) that a real scanner's specification must include – one more reason lexer-generator tools like Flex (§4.6.5) are preferred over hand-written code once a language's literal syntax grows this rich.

String-literal lexing hides similar complexity. **Escape sequences** (`\n`, `\t`, `\\`, `\uXXXX`) must be recognized *inside* the literal's pattern, not treated as ending it. **Raw strings** (Python's `r"..."`, C#'s `@"..."`) deliberately *disable* escape processing, so a single string-literal token category can need two different sub-patterns. Hardest of all is **string interpolation** / **f-strings** (Python's `f"Total = {score}"`, JavaScript template literals): the lexer must recognize where an embedded *expression* begins inside a string and, in effect, briefly hand control back to a full expression lexer/parser before resuming the string literal. This breaks the assumption that lexical analysis is a single flat pass over characters, and is a genuinely active area of parser/lexer-design discussion in modern language implementations.

4.5.4 Beyond ASCII and Flat Text: Unicode and Indentation

DB's "a letter followed by letters and digits" quietly assumes ASCII. Modern language standards (Python 3, Swift, Julia, Rust as of recent editions) permit *any* Unicode character in the "Letter"

category as part of an identifier – so accented Latin letters (`café`), non-Latin scripts (Greek, Arabic, Devanagari, Han characters), and even emoji are all valid identifiers in the languages that permit them (Swift famously allows an emoji as a variable name). Real lexer specifications reference the Unicode standard's character-category tables rather than a simple `[A-Za-z]` range.

Not every language is purely character-driven in the first place. Python and Haskell use indentation, not braces, to delimit blocks. Their lexers cannot rely on simple per-character regular patterns alone – they maintain an explicit **indentation stack** and synthesize special `INDENT` and `DEDENT` pseudo-tokens whenever a logical line's indentation increases or decreases relative to the stack's top, in addition to the ordinary tokens for the line's actual content. This is a clean example of *why* DB's model of "patterns matched against a flat character stream" is the right *starting* point but not the end of the story for every real language.

4.5.5 Seeing Real Tokenizers at Work

You do not have to take any of this on faith – most language toolchains expose their tokenizer directly. Python's `python3 -m tokenize myfile.py` prints every token, its exact source position, and its type. Clang's `clang -Xclang -dump-tokens -fsyntax-only file.c` dumps the full token stream Clang's own lexer produced. Node.js/Acorn and Babel offer their own `-tokens` flags for JavaScript. Running one of these on a file with a deliberate typo – an unterminated string, an unexpected character – is the fastest way to see exactly what a real lexer reports as an error versus what it silently accepts and passes on to the parser.

4.6 Regex in Practice

[beyond DB] The formal regular-expression notation from this lecture is the shared theoretical core behind every "regex" you have ever typed into a search box, a script, or a lexer specification – but real-world tools diverge from that core, and from each other, in important ways.

4.6.1 Not All "Regex" Is Actually Regular

This is a genuinely important and often-missed subtlety. Tools like Perl, Python's `re`, and JavaScript's regex support **backreferences** – e.g. `(\w+) \1` matches any word immediately repeated, like `hello hello`. This is provably *not* a regular language in the formal sense used in this lecture — recognizing it requires comparing two substrings for equality, which needs unbounded memory, just like the palindrome example in §4.4. Backreference-supporting engines use **backtracking search**, not finite automata, under the hood; they are more accurately called "backtracking pattern matchers" than true regular-expression engines. Lexer-generator tools like Flex, by contrast, implement only *true* regular expressions, compiled to genuine finite automata, precisely because lexical analysis needs the speed and predictability guarantees only real finite automata provide.

4.6.2 Catastrophic Backtracking (ReDoS)

Backtracking regex engines can, on adversarial input, take *exponential* time on a pattern that looks entirely innocent. The classic example is `(a+)+b` applied to a long string of `a`'s with no trailing `b`, like `"aaaaaaaaaaaaaaaaaaaaaaaaa!"`: the engine tries exponentially many ways of splitting the a's between the inner and outer `+` before giving up. This is a real, named security vulnerability class — **ReDoS** (Regular expression Denial of Service) — and has caused real production outages (Cloudflare's July 2019 global outage was triggered by exactly this kind of catastrophic backtracking in a WAF rule). The illustrative growth pattern:

Input length (a's, no trailing b)	20	25	30	35
Illustrative matching steps (order of magnitude)	$\sim 10^5$	$\sim 10^6$	$\sim 10^8$	$\sim 10^{10}$

This is exactly why DFA-based matching – which is *provably* linear in input length, with no possibility of this blow-up – is not just a theoretical nicety but the actual engineering reason production lexers

never use a naive backtracking matcher.

4.6.3 Modern Guaranteed-Linear Engines

Google's **RE2** library and Rust's `regex` crate deliberately give up backreferences and a handful of other backtracking-only features specifically so that they can guarantee linear-time matching via automata-based execution, immune to ReDoS by construction. This is the industrial embodiment of the regular-vs-non-regular distinction from §4.4: by refusing to support genuinely non-regular features, these libraries buy back the performance guarantee that true finite automata provide.

4.6.4 Brzozowski Derivatives – An Alternative to Automata

The standard way to build a matcher for a regex is to first convert it to a finite automaton (via Thompson's construction, previewed in §**??** and covered fully in the lectures ahead). There is a completely different, more recent technique worth knowing about: the **derivative** of a regular expression r with respect to a symbol a, written $\partial_a(r)$, is a new regular expression describing "what still needs to match after consuming an a." Repeatedly taking derivatives, one input symbol at a time, and checking whether the final derivative accepts ε, matches a string *without ever building an explicit automaton at all.* Janusz Brzozowski introduced this in 1964; it fell out of fashion for decades, then saw a resurgence starting around 2010 in functional-programming circles (Scala's and Racket's parser-combinator libraries use derivative-based matching), because derivatives compose beautifully with recursive data types in a way that hand-built automata do not.

4.6.5 Regular Expression Syntax in Flex

Flex, the lexer-generator tool this course uses, takes a set of regular expressions – one per token – and runs the *entire* pipeline from §**??** for you, emitting a ready-to-compile C scanner. Its regex syntax is, deliberately, close to the formal core taught in this lecture, plus a handful of practical extensions:

Flex syntax	**Meaning**	**This lecture's notation**
`r1r2`	concatenation	$r_1 r_2$
`r1|r2`	union / alternation	$r_1 \mid r_2$
`r*`	Kleene closure	r^*
`r+`	one or more	r^+
`r?`	optional	$r^?$
`(r)`	grouping	(r)
`.`	any character except newline	shorthand for $\Sigma \setminus \{\text{newline}\}$
`[abc]`, `[a-z]`	character class / range	union of the listed symbols
`[^abc]`	negated class	union of $\Sigma \setminus \{a, b, c\}$
`{n}`, `{n,m}`, `{n,}`	exactly n (or n to m, or n or more) repetitions	r concatenated with itself, that many times
`"..."`	literal string – special characters inside need no escaping	concatenation of literal symbols
`\`	escape one special character	the literal symbol itself
`{name}`	reference a named regular definition from the Definitions section	exactly d_i in a regular definition (§4.2)
`^`, `$`	anchor: only at start / end of a line	a position assertion, not part of the core theory
`r1/r2`	trailing context: match r_1 only if followed by r_2 (lookahead, not consumed)	not part of the core theory

Example: The `num` Regular Definition, Written as Flex Would See It

§4.2.2's four-piece `num` definition translates almost verbatim into a Flex specification's Definitions section:

```
digit       [0-9]
digits      {digit}+
optFrac     (\.{digits})?
optExp      (E[+-]?{digits})?
num         {digits}{optFrac}{optExp}
```

Every named piece (`digit`, `digits`, `optFrac`, `optExp`, `num`) is exactly the d_i from the formal regular definition, just written with Flex's concrete `{name}` reference syntax instead of the abstract $d_i \to r_i$ notation – the underlying regular expression is identical either way.

Exam Focus

Two Flex features above – `{n,m}` interval counts and `r1/r2` trailing context – are *not* part of the six-rule inductive definition from §4.1. Be able to explain `{n,m}` as pure syntactic sugar (exactly like r^+ or $r^?$: it expands to n to m literal copies of r concatenated), the same way §4.3 treats the other extensions – but trailing context genuinely changes *which part of the match is consumed*, which is a new capability, not just notation.

A Bit of History

The regex you type into `grep`, Python, or a Flex specification are not identical languages – they share the core ideas from this lecture but differ in surface syntax and sometimes in power. POSIX ERE (`grep -E`) stays closest to DB's formal core, with no backreferences and character classes like `[:digit:]`. PCRE, Python's `re`, and JavaScript add backreferences and lookahead-/lookbehind assertions – genuinely more expressive than regular languages, as §4.6 discussed. Knowing which flavor you're targeting matters: a pattern copied from a Python script into a Flex `.l` file may silently mean something different, or fail to compile at all.

4.7 Self-Check Questions

Practice – Not Graded

1. For the statement `total = total + rate * 60;`, write out the full stream of `<token-name, attribute-value>` pairs, following the style of Worked Examples 2 and 3.
2. Explain why `2x` (digit immediately followed by a letter) can be treated as a genuine lexical error in many languages, while `fi` cannot, even though both look like "obvious mistakes" to a human reader.
3. A lexer encounters an opening `"` with no matching closing quote before the end of the file. Describe two different, reasonable ways a lexer could recover from this and keep scanning the rest of the file.
4. Why must the longest-match (maximal munch) rule apply when scanning `i <= 10`, so that `<=` is not mistakenly split into `<` followed by `=`?
5. Explain, in your own words, why Python's lexer cannot be built from character-level patterns alone the way a C lexer can.

6. In the big-picture pipeline (§??), which single step is responsible for turning *ambiguity* (several possible next states) into a single, unambiguous next state? Name the algorithm.
7. Explain, without using the word "notation," the difference between a regular *language* and a regular *expression*.
8. Write a regular expression for all strings over $\{0, 1\}$ that *start* with `1`. Then write one for strings that *do not contain* the substring `00`.
9. Using DB's `num` regular definition from §4.2.2, trace the match for the literal `1.89E-4`, showing what each of `digits`, `optionalFraction`, and `optionalExponent` matches.
10. Rewrite r^+ and $r^?$ using only union, concatenation, and Kleene closure (no extension operators allowed).
11. Explain, in your own words and without using the word "count," why balanced parentheses of arbitrary depth cannot be recognized by any regular expression.
12. Translate the regular definition `id -> letter (letter|digit)*` into the `{name}`-based syntax a Flex Definitions section would use, matching §4.6.5's worked example.
13. A teammate writes the regex `(a|aa)+b` and complains their program hangs on long inputs with no trailing `b`. What phenomenon is this, and why does it happen with a backtracking engine but not with a DFA-based one?

Key Takeaways

- A **token** is an abstract `<name, attribute>` pair; a **pattern** is the rule describing which lexemes belong to a token; a **lexeme** is the actual matched source text. Keep these three cleanly separated.
- Lexer and parser are split into separate phases for **simplicity, efficiency, and portability** – know all three reasons.
- The lexer and parser typically interact via a pull-based `getNextToken()` call, not by materializing a full token list up front.
- Token attributes – almost always a symbol-table pointer for `ID` – are how later phases recover exactly *which* identifier or literal a token represents.
- Most apparent "typos" (like `fi` for `if`) are *not* lexical errors, because they are valid lexemes for some other token; genuine lexical errors are things no pattern can match at all (illegal characters, unterminated strings/comments, malformed numbers).
- The big picture: **regular expression → NFA → DFA → minimized DFA → transition table → lexer**, via Thompson's construction, subset construction, and minimization – every step mechanical, which is exactly why lexer generators can automate all of it.
- A **regular language** is a set of strings, defined inductively from $\emptyset$, $\{\varepsilon\}$, and $\{a\}$, closed under union, concatenation, and Kleene closure. A **regular expression** is the notation we use to name one – different concepts, same underlying class of languages.
- **Regular definitions** let you name and reuse sub-expressions; DB's `id` and `num` definitions are the two canonical worked examples to know cold.
- Extension operators (+, ?, character classes) add convenience, not expressive power – each has an exact equivalent using only the three core operators.
- Regular expressions fundamentally **cannot count** an unbounded quantity, which is exactly why balanced parentheses, nested tags, and palindromes are *not* regular – and exactly why the compiler needs a second, more powerful formalism (context-free grammars) for parsing.
- Not everything called "regex" in practice is regular in the formal sense – backreference-supporting engines use backtracking and can suffer catastrophic (ReDoS) performance; true

finite-automaton-based matching, which we build next, is immune to this by construction.

- Real-world lexers go well beyond DB's simplified patterns: the reserved-word trick, maximal munch, rich numeric/string literal formats, indentation-based tokens, and Unicode identifiers are all standard in production compilers – and Flex's own regex syntax (§4.6.5) maps almost directly onto everything formalized in this lecture.

References for This Lecture

References

[DB] Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. *Compilers: Principles, Techniques, and Tools*. 2nd ed. Pearson, 2006.

[AP] Appel, A. W. *Modern Compiler Implementation in C/Java/ML*. Cambridge University Press, 2004.

[PP] Scott, M. L. *Programming Language Pragmatics*. 4th ed. Morgan Kaufmann.